

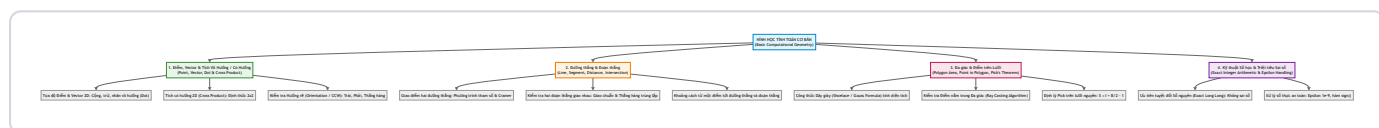
Chuyên đề: Hình học Tính toán Cơ bản (Basic Computational Geometry)

Tác giả: Duc-Minh Vu | **Đơn vị:** SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA), Đại học Kinh tế Quốc dân (NEU)

Biên soạn: Được hỗ trợ và soạn thảo bởi Agentic AI tool

Bản quyền © 2026 Duc-Minh Vu. Mọi quyền được bảo lưu. Tài liệu đào tạo và bồi dưỡng sinh viên Olympic Tin học / ICPC.

Sơ đồ Tư duy & Kiến trúc Chuyên đề



1. Điểm, Vector & Các Phép toán Cốt lõi

Trong mặt phẳng $\mathbb{R}^2$, một điểm hoặc vector được biểu diễn bằng cặp tọa độ (x, y) .

1.1. Cấu trúc Điểm & Vector Chuẩn mực

```
template <typename T = long long>
struct Point {
    T x, y;
    Point(T x = 0, T y = 0) : x(x), y(y) {}

    Point operator+(const Point& o) const { return Point(x + o.x, y + o.y); }
    Point operator-(const Point& o) const { return Point(x - o.x, y - o.y); }
    Point operator*(T scalar) const { return Point(x * scalar, y * scalar); }
    Point operator/(T scalar) const { return Point(x / scalar, y / scalar); }

    bool operator==(const Point& o) const { return x == o.x && y == o.y; }
    bool operator<(const Point& o) const {
        if (x != o.x) return x < o.x;
        return y < o.y;
    }
};
```

1.2. Tích Vô hướng (Dot Product)

Cho hai vector $\vec{u} = (x_1, y_1)$ và $\vec{v} = (x_2, y_2)$:

$$\vec{u} \cdot \vec{v} = x_1x_2 + y_1y_2 = |\vec{u}| \cdot |\vec{v}| \cdot \cos(\theta)$$

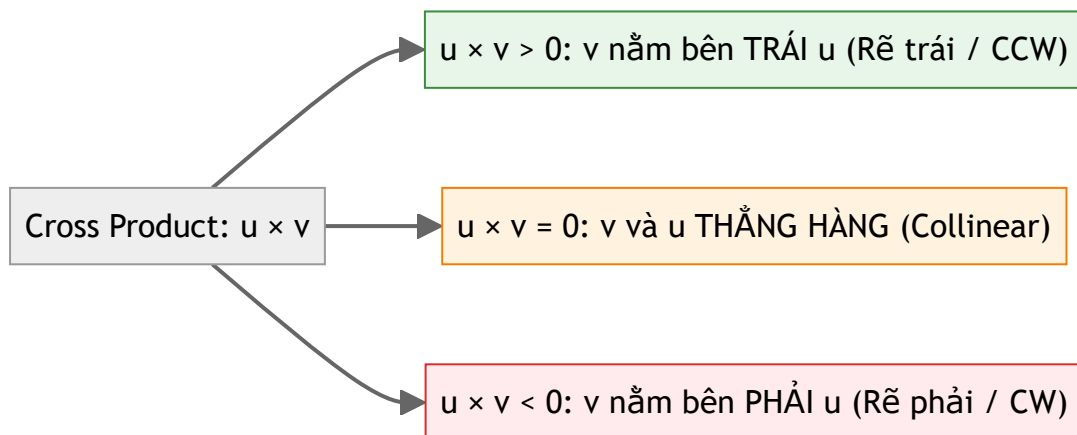
- $\vec{u} \cdot \vec{v} > 0 \iff$ Góc θ giữa hai vector là **góc nhọn** ($\theta < 90^\circ$).
- $\vec{u} \cdot \vec{v} = 0 \iff$ Hai vector **vuông góc** ($\theta = 90^\circ$).
- $\vec{u} \cdot \vec{v} < 0 \iff$ Góc θ giữa hai vector là **góc tù** ($\theta > 90^\circ$).
- Bình phương độ dài: $|\vec{u}|^2 = \vec{u} \cdot \vec{u} = x_1^2 + y_1^2$.

```
template <typename T>
T dot(Point<T> a, Point<T> b) {
    return a.x * b.x + a.y * b.y;
}
```

1.3. Tích Có hướng 2D (Cross Product)

Trong không gian 2D, tích có hướng của hai vector $\vec{u} = (x_1, y_1)$ và $\vec{v} = (x_2, y_2)$ là một đại lượng vô hướng có hướng:

$$\vec{u} \times \vec{v} = \det \begin{pmatrix} x_1 & x_2 \\ y_1 & y_2 \end{pmatrix} = x_1y_2 - x_2y_1 = |\vec{u}| \cdot |\vec{v}| \cdot \sin(\theta)$$



✦ Ý nghĩa hình học:

- $|\vec{u} \times \vec{v}|$ chính là **Diện tích hình bình hành** tạo bởi hai vector $\vec{u}$ và $\vec{v}$.
- Diện tích tam giác tạo bởi $O, \vec{u}, \vec{v}$ là $S = \frac{1}{2}|\vec{u} \times \vec{v}|$.

```
template <typename T>
T cross(Point<T> a, Point<T> b) {
    return a.x * b.y - a.y * b.x;
}

// Tích có hướng của hai vector AB và AC xuất phát từ A
template <typename T>
T cross(Point<T> a, Point<T> b, Point<T> c) {
    return cross(b - a, c - a);
}
```

1.4. Hàm Hướng rẽ (Orientation / CCW Test)

Cho 3 điểm A, B, C . Xác định hướng rẽ của đường gấp khúc $A \rightarrow B \rightarrow C$:

$$O(A, B, C) = \vec{AB} \times \vec{AC} = (B.x - A.x)(C.y - A.y) - (B.y - A.y)(C.x - A.x)$$

- $O(A, B, C) > 0$: Rẽ trái (Counter-Clockwise - Ngược chiều kim đồng hồ).
- $O(A, B, C) < 0$: Rẽ phải (Clockwise - Cùng chiều kim đồng hồ).
- $O(A, B, C) = 0$: Ba điểm A, B, C thẳng hàng (Collinear).

```
template <typename T>
int ccw(Point<T> a, Point<T> b, Point<T> c) {
    T cp = cross(a, b, c);
    if (cp > 0) return 1;    // Rẽ trái (CCW)
    if (cp < 0) return -1;   // Rẽ phải (CW)
    return 0;               // Thẳng hàng
}
```

2. Đường thẳng & Đoạn thẳng (Lines & Segments)

2.1. Kiểm tra Điểm nằm trên Đoạn thẳng (Point on Segment)

Điểm P nằm trên đoạn thẳng AB khi và chỉ khi:

1. P, A, B thẳng hàng: $\vec{AP} \times \vec{AB} = 0$.
2. P nằm trong hình hộp giới hạn (Bounding Box) của AB :

$$\min(A.x, B.x) \leq P.x \leq \max(A.x, B.x) \quad \text{và} \quad \min(A.y, B.y) \leq P.y \leq \max(A.y, B.y)$$

```
template <typename T>
bool onSegment(Point<T> p, Point<T> a, Point<T> b) {
    if (cross(a, b, p) != 0) return false;
    return (p.x >= min(a.x, b.x) && p.x <= max(a.x, b.x) &&
            p.y >= min(a.y, b.y) && p.y <= max(a.y, b.y));
}
```

2.2. Kiểm tra Hai Đoạn thẳng Cắt nhau (Segment-Segment Intersection)

Cho hai đoạn thẳng AB và CD . Chúng cắt nhau khi và chỉ khi thỏa mãn một trong hai trường hợp:

Trường hợp 1: Cắt nhau thông thường (Proper Intersection)

- Hai điểm C và D nằm về hai phía khác nhau so với đường thẳng AB :

$$\text{ccw}(A, B, C) \cdot \text{ccw}(A, B, D) < 0$$

- VÀ hai điểm A và B nằm về hai phía khác nhau so với đường thẳng CD :

$$\text{ccw}(C, D, A) \cdot \text{ccw}(C, D, B) < 0$$

Trường hợp 2: Cắt nhau suy biến (Collinear Overlapping)

- Bốn điểm thẳng hàng hoặc một đầu mút nằm trên đoạn thẳng kia:
- C nằm trên đoạn AB , HOẶC
- D nằm trên đoạn AB , HOẶC
- A nằm trên đoạn CD , HOẶC
- B nằm trên đoạn CD .

```
template <typename T>
bool intersect(Point<T> a, Point<T> b, Point<T> c, Point<T> d) {
    int o1 = ccw(a, b, c);
    int o2 = ccw(a, b, d);
    int o3 = ccw(c, d, a);
    int o4 = ccw(c, d, b);

    // Trường hợp thông thường: hai đầu mút đối xứng qua hai đường thẳng
    if (o1 * o2 < 0 && o3 * o4 < 0) return true;

    // Trường hợp suy biến: một điểm nằm trên đoạn thẳng kia
    if (o1 == 0 && onSegment(c, a, b)) return true;
    if (o2 == 0 && onSegment(d, a, b)) return true;
    if (o3 == 0 && onSegment(a, c, d)) return true;
    if (o4 == 0 && onSegment(b, c, d)) return true;

    return false;
}
```

2.3. Tìm Giao điểm Hai Đường thẳng (Line Intersection)

Hai đường thẳng đi qua (A, B) và (C, D) :

$$P = A + \vec{AB} \cdot t, \quad \text{với } t = \frac{\vec{AC} \times \vec{CD}}{\vec{AB} \times \vec{CD}}$$

Nếu $\vec{AB} \times \vec{CD} = 0$, hai đường thẳng song song hoặc trùng nhau.

```

Point<double> lineIntersection(Point<double> a, Point<double> b, Point<double> c, Point<double> d) {
    Point<double> ab = b - a;
    Point<double> cd = d - c;
    Point<double> ac = c - a;
    double cp = cross(ab, cd);
    assert(abs(cp) > 1e-9); // Không song song
    double t = cross(ac, cd) / cp;
    return a + ab * t;
}

```

3. Đa giác (Polygons)

3.1. Diện tích Đa giác: Công thức Dây giày (Shoelace Formula)

Cho đa giác đơn N đỉnh $P_0, P_1, \dots, P_{N-1}$ liệt kê theo chiều liên tiếp (cùng chiều hoặc ngược chiều kim đồng hồ).

Diện tích có hướng (Signed Area):

$$2 \cdot S = \sum_{i=0}^{N-1} (P_i \times P_{i+1}) = \sum_{i=0}^{N-1} (x_i y_{i+1} - x_{i+1} y_i) \quad (\text{với } P_N = P_0)$$

Diện tích thực sự là $S = \frac{1}{2} |2S|$.

```

template <typename T>
T polygonArea2(const vector<Point<T>>& poly) {
    int n = poly.size();
    T sum = 0;
    for (int i = 0; i < n; ++i) {
        int j = (i + 1) % n;
        sum += cross(poly[i], poly[j]);
    }
    return abs(sum); // Trả về 2 * Area (số nguyên chính xác)
}

```

3.2. Kiểm tra Điểm nằm trong Đa giác (Point in Polygon)

Cho đa giác đơn N đỉnh và điểm kiểm tra Q .

Thuật toán Chiếu tia (Ray Casting Algorithm):

Phóng một tia ngang từ Q sang phải đến $+\infty$ ($y = Q.y, x \geq Q.x$).

Đếm số lần tia cắt các cạnh của đa giác:

- **Nằm trên biên (BOUNDARY):** Nếu Q nằm trên bất kỳ cạnh nào của đa giác.
- **Nằm bên trong (INSIDE):** Nếu số giao điểm là **số lẻ**.
- **Nằm bên ngoài (OUTSIDE):** Nếu số giao điểm là **số chẵn**.

💡 Quy tắc xử lý biên chuẩn mực:

Cạnh AB cắt tia ngang khi một đầu mút nằm dưới tia ($A.y \leq Q.y$) và đầu mút kia nằm trên tia ($B.y > Q.y$), và giao điểm nằm bên phải Q .

```
// 0: OUTSIDE, 1: BOUNDARY, 2: INSIDE
template <typename T>
int pointInPolygon(Point<T> q, const vector<Point<T>>& poly) {
    int n = poly.size();
    bool inside = false;

    for (int i = 0; i < n; ++i) {
        Point<T> a = poly[i];
        Point<T> b = poly[(i + 1) % n];

        // Kiểm tra xem Q có nằm trên cạnh AB không
        if (onSegment(q, a, b)) return 1; // BOUNDARY

        // Chuẩn hóa chiều dọc để a.y <= b.y
        if (a.y > b.y) swap(a, b);

        // Kiểm tra xem tia ngang từ Q sang phải có cắt đoạn thẳng AB không
        if (a.y <= q.y && q.y < b.y) {
            // Tích có hướng xét vị trí của Q so với vector AB
            if (cross(a, b, q) > 0) {
                inside = !inside;
            }
        }
    }

    return inside ? 2 : 0; // 2: INSIDE, 0: OUTSIDE
}
```

3.3. Định lý Pick trên Lưới Nguyên

Cho một đa giác đơn có tất cả các đỉnh đều nằm trên lưới điểm nguyên $\mathbb{Z}^2$.

Gọi:

- S : Diện tích của đa giác (tính bằng công thức Shoelace).
- I : Số điểm nguyên nằm **hoàn toàn bên trong** đa giác.
- B : Số điểm nguyên nằm **trên cạnh/biên** của đa giác.

🏆 Định lý Pick:

$$S = I + \frac{B}{2} - 1 \implies I = S - \frac{B}{2} + 1$$

Cách tính số điểm nguyên B trên biên:

Số điểm nguyên trên đoạn thẳng nối hai điểm nguyên (x_1, y_1) và (x_2, y_2) (không tính đầu mút cuối):

$$B_{\text{segment}} = \gcd(|x_2 - x_1|, |y_2 - y_1|)$$

Tổng điểm biên:

$$B = \sum_{i=0}^{N-1} \gcd(|x_{i+1} - x_i|, |y_{i+1} - y_i|)$$

4. Kỹ thuật Xử lý Sai số trong Lập trình Thi đấu

4.1. Nguyên tắc Vàng: Ưu tiên Số nguyên Tuyệt đối

Bất cứ khi nào có thể, hãy dùng `long long` hoặc `__int128`:

- Tránh phép chia: So sánh $\frac{a}{b} < \frac{c}{d} \iff a \cdot d < b \cdot c$ (với $b, d > 0$).
- Tránh căn bậc hai: So sánh khoảng cách $AB < CD \iff AB^2 < CD^2$.
- Dùng $2 \times S$ (số nguyên) thay vì S (số thực).

4.2. Xử lý Sai số Số thực khi bắt buộc

```
const double EPS = 1e-9;

inline int sign(double x) {
    if (x > EPS) return 1;
    if (x < -EPS) return -1;
    return 0;
}

inline bool dEqual(double a, double b) {
    return abs(a - b) < EPS;
}
```

5. Chạm bẫy Phòng thi Thường gặp

STT	Chạm bẫy thường gặp	Giải pháp phòng ngừa
1	Tràn số khi tính tích có hướng	Tọa độ $10^9 \implies$ tích chéo lên đến 2×10^{18} , chạm ngưỡng <code>long long</code> . Ép kiểu <code>__int128</code> hoặc <code>long double</code> khi cần.
2	Bỏ quên trường hợp các điểm thẳng hàng	Hàm <code>intersect</code> luôn phải bao gồm kiểm tra <code>onSegment</code> cho các trường hợp suy biến $\sigma = 0$.
3	Tia ngang đi qua đúng một đỉnh của đa giác	Dùng điều kiện $a.y \leq Q.y < b.y$ (nửa khoảng đóng - mở) để mỗi đỉnh chỉ được đếm đúng một lần.

STT	Cạm bẫy thường gặp	Giải pháp phòng ngừa
4	Đa giác tự cắt (Self-intersecting)	Công thức Shoelace chỉ áp dụng cho đa giác đơn (Simple Polygon) không tự cắt.



Competitive Programming Handbook

SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA) — National Economics University (NEU)

Tài liệu được soạn thảo và tối ưu bởi Agentic AI tool

© 2026 Duc-Minh Vu. Toàn bộ bản quyền được bảo lưu.